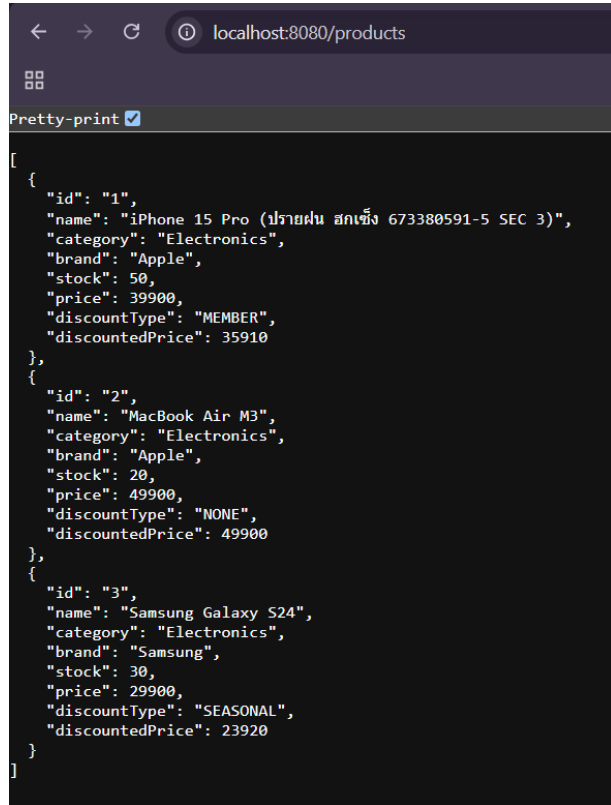


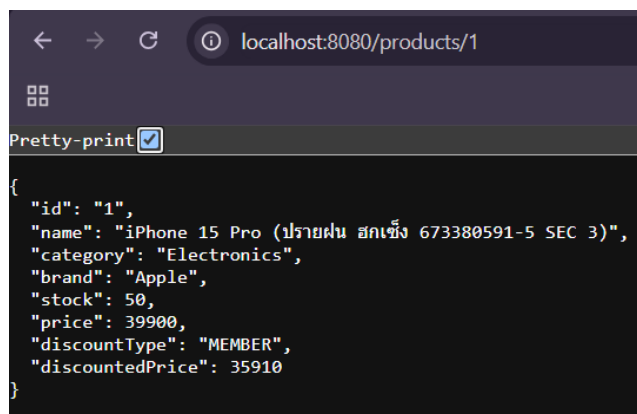
Lab10

GET <http://localhost:8080/products>



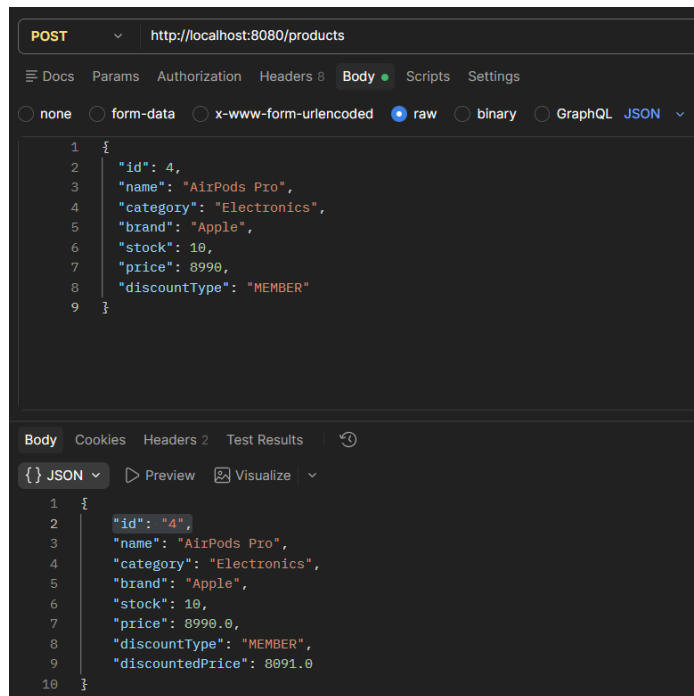
```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (ปรายฝน สกแข็ง 673380591-5 SEC 3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

GET [/products/{id}](http://localhost:8080/products/{id})

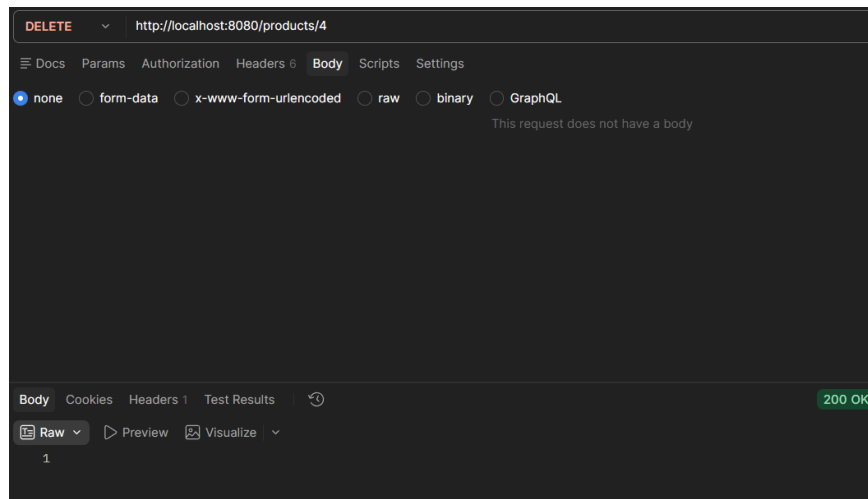


```
{
  "id": "1",
  "name": "iPhone 15 Pro (ปรายฝน สกแข็ง 673380591-5 SEC 3)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900,
  "discountType": "MEMBER",
  "discountedPrice": 35910
}
```

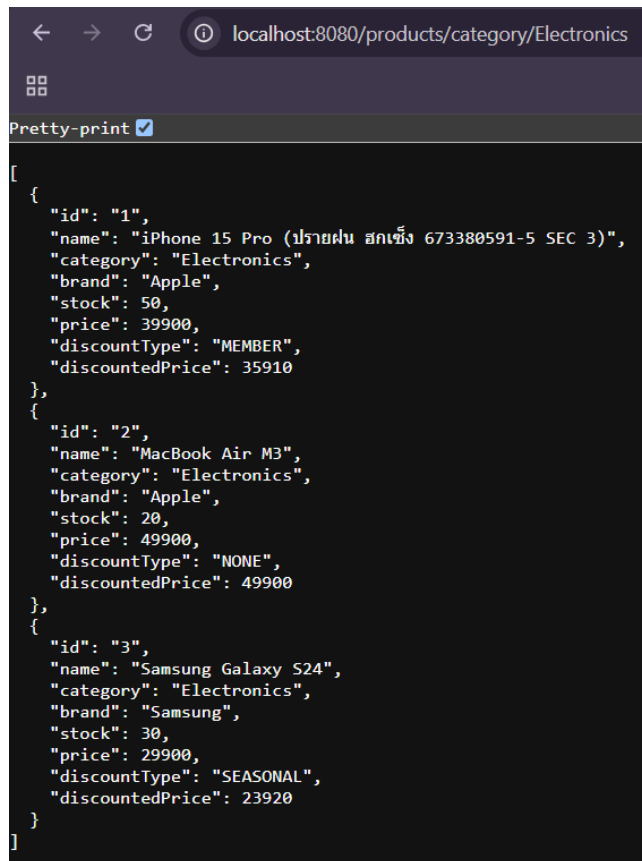
POST /products



DELETE /products/4

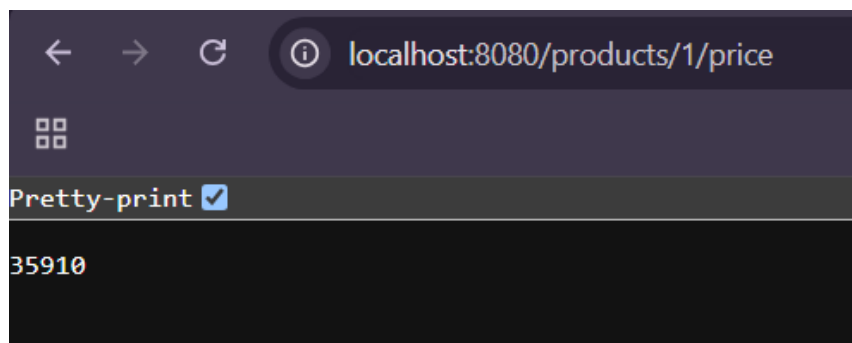


GET /products/category/Electronics



```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (ปรายณณ สกเซ็ง 673380591-5 SEC 3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

GET /products/1/price



```
35910
```

คำถามท้ายLab

1.Reactive Programming vs Blocking — ต่างกันอย่างไร

Blocking คือการทำงานที่ต้องรอให้คำสั่งหนึ่งทำงานเสร็จก่อน จึงจะไปทำงานต่อ ทำให้ Thread ต้องรอผลลัพธ์ ส่วน Reactive Programming คือการทำงานแบบไม่ต้องรอผลลัพธ์ ทำให้ Thread สามารถไปทำงานอื่นต่อได้ จึงเหมาะกับงานที่ต้องรับส่งข้อมูลหรือทำงานหลายอย่างพร้อมกัน

2.Mono vs Flux — ใช้กรณีไหน และทำไม

Mono ใช้เมื่อเราต้องการผลลัพธ์ 0 หรือ 1 ค่า เช่น การค้นหา Product จาก ID

ส่วน Flux ใช้เมื่อเราต้องการผลลัพธ์ หลายค่า เช่น การดึง Product ทั้งหมดหรือค้นหาสินค้าตาม Category

3.WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

.get() ใช้ส่งคำขอแบบ GET

.uri() กำหนด URL ที่ต้องการเรียก

.retrieve() รับข้อมูล Response กลับมา

.bodyToMono() แปลงข้อมูลเป็น Mono

.bodyToFlux() แปลงข้อมูลเป็น Flux

4.Code ของ Repository, Service, Controller พร้อม comment อธิบาย

Repository ใช้สำหรับจัดการข้อมูล Product เช่น ค้นหา เพิ่ม ลบ และค้นหาตาม Category

```
// ค้นหา Product จาก ID
public Mono<Product> findById(String id) {
    Product product = store.get(id);
    return product == null ? Mono.empty() : Mono.just(product);
}

// ดึง Product ทั้งหมด
public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

// เพิ่มหรือบันทึก Product
public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

// ลบ Product ตาม ID
public Mono<Void> deleteById(String id) {
    store.remove(id);
    return Mono.empty();
}

// ค้นหา Product ตาม Category
public Flux<Product> findByCategory(String category) {
    return findAll()
        .filter(p -> p.getCategory().equalsIgnoreCase(category));
}
```

Service ทำหน้าที่เป็นตัวกลางระหว่าง Controller กับ Repository และจัดการ Logic ของระบบ

```
// ค้นหา Product จาก ID
public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(Mono.error(
            new RuntimeException("Product not found: " + id)
        ));
}

// ดึง Product ทั้งหมด
public Flux<Product> getAll() {
    return repository.findAll();
}

// บันทึก Product และสร้าง ID อัตโนมัติถ้ายังไม่มี
public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(java.util.UUID.randomUUID().toString());
    }
    return repository.save(product);
}

// ลบ Product
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

// ค้นหาตาม Category
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// คำนวณราคาหลังหักส่วนลด
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id)
        .map(p -> p.getDiscountedPrice());
}
```

Controller ทำหน้าที่รับ Request จากผู้ใช้ แล้วเรียก Service เพื่อประมวลผลและส่ง Response กลับไป

```
// GET /products/{id}
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    return service.getById(id);
}

// GET /products
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}

// POST /products
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}

// DELETE /products/{id}
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

// GET /products/category/{category}
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

// GET /products/{id}/price
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}
```

